ETEC PROFª ERMELINDA GIANNINI TEIXEIRA
ENSINO MÉDIO INTEGRADO AO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

João Victor Silva Farias
Luan De Jesus Almeida
Matheus Henrique Sá Teles Bertunes
Talita Lima Ferreira
Vitor Amorim Spaolonse

MANUAL DO DESENVOLVEDOR

BXVS Connect: Sistema de gerenciamento de academia

SANTANA DE PARNAÍBA
Setembro de 2025

SUMÁRIO

1. INTRODUÇÃO 2
2. ARQUITETURA DO SISTEMA 3
Tecnologias Utilizadas 3
3. REQUISITOS DO SISTEMA 3
3.1. Requisitos Funcionais 4
3.2. Requisitos Não Funcionais 4
4. CONFIGURAÇÃO DO AMBIENTE DE DESENVOLVIMENTO 4
4.1. Pré-requisitos 4
4.2. Instalação 4
4.3. Configuração da API de Reconhecimento Facial 5
4.4. Integração com Mercado Pago 5
5. ESTRUTURA DO BANCO DE DADOS 5
6. MÓDULOS PRINCIPAIS 7
6.1. Reconhecimento Facial 7
6.2. Pagamento Online 7
6.3. Dashboard Administrativo 7
6.4. Gestão de Alunos 7
7. DIAGRAMAS 7
7.1. Diagrama de Casos de Uso 8
7.2. Diagrama de Classes 8
8. DEPLOY 8
8.1. Ambiente de Produção 9
8.2. Variáveis de Ambiente 9

1. INTRODUÇÃO

O **BXVS Connect** é um sistema integrado de gestão para academias, desenvolvido para automatizar processos como check-in, controle de frequência, gestão de pagamentos, comunicação com alunos e geração de relatórios administrativos. O sistema foi projetado para atender às necessidades específicas da academia **Box Vida Saudável (BXVS)**, localizada em Santana de Parnaíba-SP, e utiliza tecnologias modernas para garantir eficiência, segurança e escalabilidade.

2. ARQUITETURA DO SISTEMA

O sistema é composto por dois ambientes principais:

- **Sistema de Gestão (Backoffice):** Acesso restrito a funcionários e administradores
- **Site de Inscrição (Portal do Aluno):** Interface pública para cadastro, pagamento e consulta de dados

Tecnologias Utilizadas

Componente	Tecnologia/Estrutura
Front-end	React, JavaScript, HTML5, CSS3
Back-end	Python com Flask
Banco de Dados	MySQL
Reconhecimento Facial	Python, Face API JS
Pagamentos	API do Mercado Pago
Versionamento	Git
IDE	Visual Studio Code

3. REQUISITOS DO SISTEMA

3.1. Requisitos Funcionais

Código	Função	Classificação	Visibilidade	Descrição
RF001	Gerenciar alunos		Evidente	**Administrador** pode cadastrar, editar, excluir e consultar dados dos alunos
RF002	Relatório de presença	Obrigatório	Evidente	**Administrador** tem acesso a um panorama de assiduidade e presença diária
RF003	Repassar pagamentos	Obrigatório	Oculto	O **sistema** deve repassar pagamentos dos planos para a API
RF004	Gerenciar pagamentos	Obrigatório	Evidente	**Administrador** tem um Panorama de pagamentos realizados
RF006	Check-in	Desejável	Evidente	Permitir check-in de alunos com reconhecimento facial a partir de API
RF005	Reconhecer o rosto	Desejável	Oculto	O **sistema** deve ligar o rosto da API para o banco de dados já pré-cadastrado pelo usuário
RF007	Cadastrar dados	Obrigatório	Evidente	**Usuário** deverá ser capaz de cadastrar seus dados no sistema
RF008	Realizar pagamentos	Obrigatório	Evidente	**Usuário** deverá cadastrar seus dados de pagamento para ligação à API
RF009	Visualizar frequência	Desejável	Evidente	**Usuário** terá acesso a seu estado de frequência

3.2. Requisitos Não Funcionais

- **Usabilidade:** Interface intuitiva e responsiva
- **Segurança:** Criptografia de dados e autenticação facial
- **Desempenho:** Processamento rápido de check-in e relatórios
- **Escalabilidade:** Suporte a crescimento no número de usuários

4. CONFIGURAÇÃO DO AMBIENTE DE DESENVOLVIMENTO

4.1. Pré-requisitos

- Python 3.8+
- Node.js e npm
- MySQL 8.0+
- Git

4.2. Instalação

1. **Clone o repositório:**

git clone https://github.com/bxvs-connect/sistema.git

2. **Front-end (React):**

cd frontend
npm install
npm start

3. **Banco de Dados:**

- Importe o script database.sql no MySQL
- Configure as variáveis de ambiente no arquivo .env

4.3. Configuração da API de Reconhecimento Facial

- Instale as bibliotecas:pip install opencv-python face-recognition flask-cors

- Configure a câmera e o caminho para o modelo de reconhecimento

4.4. Integração com Mercado Pago

- Crie uma conta no Mercado Pago Developers
- Adicione as credenciais no arquivo de configuração do back-end

5. ESTRUTURA DO BANCO DE DADOS

```
create database academia;
use academia;

CREATE TABLE administrador(
  idProfessor INT primary key AUTO_INCREMENT,
  nome VARCHAR(200) NOT NULL,
  email VARCHAR(200) unique NOT NULL,
  senha VARCHAR(20) NOT NULL
);

CREATE TABLE cliente(
  idCliente INT primary key auto_increment,
  nome VARCHAR(200) NOT NULL,
  email VARCHAR(200) unique NOT NULL,
  senha VARCHAR(20) NOT NULL,
  telefone varchar(20) NOT NULL,
  sexo VARCHAR(9) NOT NULL,
  rg varchar(20) NOT NULL,
  cpf varchar(20) NOT NULL,
  dataNascimento DATE NOT NULL,
```

```
  estadoCivil VARCHAR(20) NOT NULL,
  dataCadastro datetime NOT NULL
  );

CREATE TABLE plano(
  idPlano INT primary key AUTO_INCREMENT,
  nome VARCHAR(45) NOT NULL,
  descricao TEXT NOT NULL,
  preco DECIMAL(10,2) NOT NULL,
  duracaoMeses INT NOT NULL
  );

CREATE TABLE matricula (
  idMatricula INT primary key AUTO_INCREMENT,
  dataInicio DATE NOT NULL,
  dataFim DATE NOT NULL,
  _status  ENUM("ativa", "inativa", "pendente") NOT NULL,
  idCliente INT NOT NULL,
  idPlano INT NOT NULL,
  FOREIGN KEY (idCliente) REFERENCES cliente(idCliente),
  FOREIGN KEY (idPlano) REFERENCES plano(idPlano)
);

CREATE TABLE pagamento (
  idPagamento INT primary key AUTO_INCREMENT,
  valor DECIMAL(10,2) NOT NULL,
  dataPagamento TIMESTAMP NOT NULL DEFAULT current_timestamp,
  metodo ENUM("dinheiro", "cartão", "pix", "boleto") NOT NULL,
  _status ENUM("pago", "pendente", "atrasado") NOT NULL DEFAULT "pendente",
  idMatricula INT NOT NULL,
  FOREIGN KEY (idMatricula) REFERENCES matricula(idMatricula)
);
```

```
select * from cliente
```

6. MÓDULOS PRINCIPAIS

6.1. Reconhecimento Facial

- Desenvolvido em Python com Flask e Face API JS
- Processo:
    - Captura da imagem
    - Comparação com base de dados
    - Retorno do ID do aluno para check-in

6.2. Pagamento Online

- Integração com a API do Mercado Pago
- Fluxo:
    - Seleção do plano no site
    - Redirecionamento para o Mercado Pago
    - Retorno com confirmação de pagamento
    - Ativação automática do plano

6.3. Dashboard Administrativo

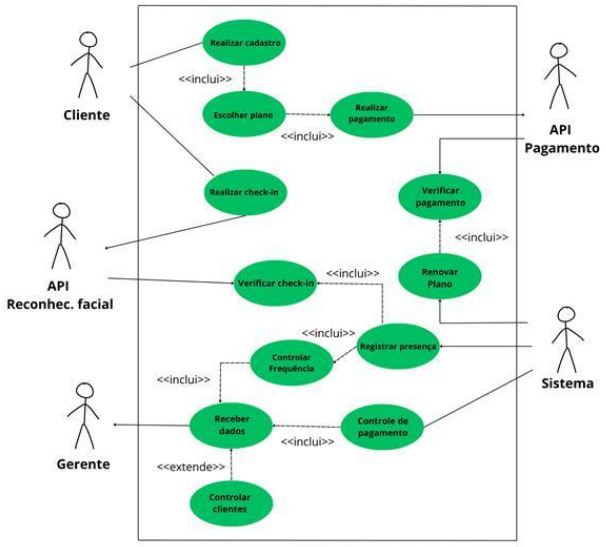
- Gráficos e métricas em tempo real
- Receita, frequência, inadimplência, etc.
- Filtros por data e turno

6.4. Gestão de Alunos

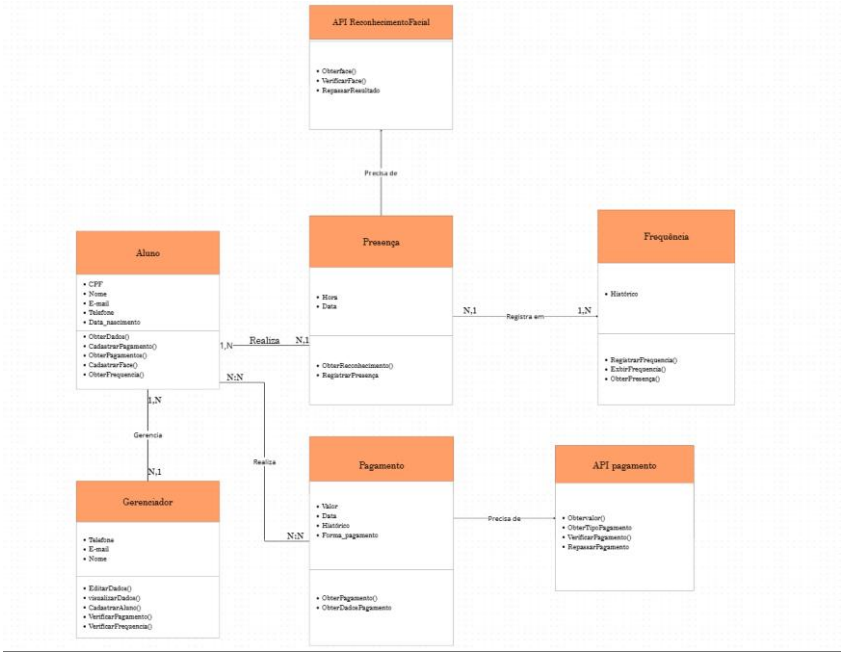
- Cadastro, edição, exclusão
- Envio de lembretes de pagamento
- Filtros e relatórios

7. DIAGRAMAS

7.1. Diagrama de Casos de Uso



7.2. Diagrama de Classes



8. DEPLOY

8.1. Ambiente de Produção

- **Front-end:** Hospedado em servidor web Vercel(ex: Netlify, Vercel)
- **Back-end:** Deploy em servidor com Pythonanywhe e Flask (ex: Heroku, AWS)
- **Banco de Dados:** MySQL em servidor cloud (ex: AWS RDS, Azure SQL)

8.2. Variáveis de Ambiente

Configure no servidor:

```
DB_HOST=...
DB_USER=...
DB_PASSWORD=...
MERCADO_PAGO_ACCESS_TOKEN=...
FACE_RECOGNITION_MODEL_PATH=...
```